

Wanderlust Project: Interview Notes

A study guide with clear definitions and ready answers for the Wanderlust listing app (Node.js, Express, MongoDB, Mongoose, EJS).

1. Project Introduction

Short pitch (30-40 seconds):

Wanderlust is a full-stack listing application similar to Airbnb. I built it with Node.js, Express, MongoDB (Mongoose) and EJS for server-side rendering. Users can view all listings, open a single listing, create, edit and delete listings, so it covers full CRUD. I used RESTful routes, method-override for PUT and DELETE, and ejs-mate for reusable layouts. Next, I plan to add validation, error handling, authentication and image upload.

Tech stack

Layer	Technology	Purpose
Runtime	Node.js	Runs JavaScript on the server
Framework	Express	Routing and middleware
Database	MongoDB	Stores listings as documents
ODM	Mongoose	Schema and model layer over MongoDB
View engine	EJS + ejs-mate	Server-side HTML templates with layouts
Helper	method-override	Enables PUT and DELETE from HTML forms

2. Core Definitions

Node.js

A JavaScript runtime built on Chrome's V8 engine that lets JavaScript run outside the browser. It uses an event-driven, non-blocking I/O model, which makes it good for handling many concurrent requests.

Express.js

A minimal web framework for Node.js. It provides routing, middleware support, and helpers for handling requests and responses.

Middleware

A function that runs between the incoming request and the final response. It receives `req`, `res` and `next`. It can modify the request or response, end the request-response cycle, or pass control forward by calling `next()`.

Used in this project:

- `express.urlencoded({ extended: true })` parses form data into `req.body`
- `method-override` converts POST with `?_method=PUT` into a PUT request
- `express.static` serves files from the `public` folder

Routing

Mapping an HTTP method and URL path to a handler function. Example: `GET /listing/:id` runs the show handler.

REST (Representational State Transfer)

An architectural style where URLs represent resources and HTTP methods represent actions on them.

Action	Method	Route
Read all	GET	<code>/listings</code>
Read one	GET	<code>/listings/:id</code>
Create	POST	<code>/listings</code>
Update	PUT	<code>/listings/:id</code>
Delete	DELETE	<code>/listings/:id</code>

CRUD

Create, Read, Update, Delete: the four basic operations on stored data.

`req.params`, `req.query`, `req.body`

- `req.params`: values from the route path, e.g. `:id` in `/listing/:id`
- `req.query`: values from the URL query string, e.g. `?search=goa`
- `req.body`: data sent in the request body, e.g. form fields on POST

PUT vs PATCH

- **PUT** replaces the entire resource.
- **PATCH** updates only the fields sent.

method-override

HTML forms only support GET and POST. `method-override` lets a form send a POST with `?_method=PUT` or `?_method=DELETE`, and converts it to the intended method before it reaches the route.

Route order

Express matches routes top to bottom. `/listing/new` must be defined before `/listing/:id`, otherwise "new" is treated as an `:id` value.

3. MongoDB and Mongoose

MongoDB

A NoSQL document database. Data is stored as flexible JSON-like documents (BSON) inside collections, instead of rows inside tables.

SQL vs NoSQL (why MongoDB here)

Listings are document-shaped (title, description, image, price, location), need no complex joins, and the schema may change as features are added. MongoDB stores this naturally and pairs well with JavaScript.

Mongoose

An ODM (Object Data Modeling) library for MongoDB and Node.js. It adds schemas, validation, and easy query methods.

Schema

Defines the structure of a document: field names, types, defaults and validation rules.

Model

A class compiled from a schema. It gives methods to interact with the collection, such as `find`, `findById`, `save`. In this project, `Listing` is the model.

Query methods

Method	Behavior
<code>find({})</code>	Returns an array of all matching documents
<code>findOne(cond)</code>	Returns the first matching document
<code>findById(id)</code>	Returns one document by <code>_id</code>
<code>findByIdAndUpdate(id, data)</code>	Updates and returns the old document by default; use <code>{ new: true }</code> to get the updated one
<code>findByIdAndDelete(id)</code>	Deletes and returns the deleted document

Index

A data structure that speeds up queries on a field (for example `location` or `price`), at the cost of extra storage and slower writes.

Embedding vs Referencing

- **Embedding:** store related data inside the same document. Good for data read together.
- **Referencing:** store an `ObjectId` pointing to another document. Good for data that grows or is shared, such as reviews attached to a listing.

4. Async JavaScript

Promise

An object representing a value that will be available later (pending, fulfilled or rejected).

async / await

Syntax for working with promises in a sequential style. `await` pauses execution inside an `async` function until the promise settles, without blocking the whole server.

Why database calls are async

Database operations take time (network and disk). Async code lets Node.js keep serving other requests while waiting.

5. EJS and Rendering

EJS (Embedded JavaScript templates)

A template engine that lets you embed JavaScript in HTML to generate dynamic pages on the server.

Server-Side Rendering (SSR)

The server builds the full HTML using database data and sends it to the browser. Good for SEO and simple pages. The alternative is **Client-Side Rendering (CSR)**, where the browser builds the page with JavaScript (e.g. React).

`<%= %>` vs `<%- %>`

- `<%= %>` outputs escaped HTML (safe against XSS)
- `<%- %>` outputs raw, unescaped HTML (use only for trusted content such as layout includes)

ejs-mate

Adds layout support to EJS, so shared parts like the navbar and footer are written once and reused on every page.

6. Ready Answers

Why method-override?

HTML forms only support GET and POST. For update and delete I need PUT and DELETE, so I send a POST with `?_method=PUT` and method-override converts it before it reaches my route.

Why MongoDB?

Listings are flexible, document-like data with no complex joins, and MongoDB works naturally with JavaScript across the stack.

What does `findByIdAndUpdate` return?

By default the old document. To get the updated one I pass `{ new: true }`.

Why is `/listing/new` above `/listing/:id`?

Express matches routes in order. Otherwise "new" would be captured as an id.

What is middleware?

A function with access to `req`, `res` and `next` that runs during the request cycle. I used it for parsing form data, method override and serving static files.

7. Known Gaps and Improvements

Be honest about these, and explain how you would fix them.

Gap	Fix
No error handling; a bad id or DB failure can crash the app	<code>wrapAsync</code> helper plus a central error-handling middleware (<code>err, req, res, next</code>)
No server-side validation	Joi schema validation and Mongoose validators
Inconsistent route names (<code>/listing</code> vs <code>/listings</code>)	Use one plural, RESTful convention everywhere
Everything in one file	Split into <code>express.Router()</code> files and controllers (MVC)
Hardcoded DB URL and port	Environment variables with <code>dotenv</code>
<code>findById</code> may return <code>null</code>	Check for not found and respond with a 404
Image stored only as a URL string	Multer plus Cloudinary for real uploads

Sample answer to "What would you improve?"

First, error handling with a `wrapAsync` helper and central error middleware. Second, validation with Joi so invalid data never reaches the database. Third, better structure by moving routes into routers and controllers. After that, authentication, image upload, and reviews as a second collection referencing the listing.

8. Follow-up Topics

Authentication (how to add)

Create a `User` model, hash passwords with `bcrypt`, start a session or issue a JWT on login, and protect create, edit and delete routes with an `isLoggedInIn` middleware. Add an `owner` field to each listing so only the owner can modify it.

Deployment

Use MongoDB Atlas for the database, store secrets in environment variables, and deploy the app on a platform such as Render or Railway.

Search and pagination

Use query strings (`?search=`, `?page=`) with Mongoose filters, `skip()` and `limit()`, and add an index on the searched fields.

Reviews feature

Create a `Review` schema with a rating and comment, store review ids in the listing (or the listing id in the review), and use `populate()` to load them.

9. Delivery Tips

1. Structure answers as: what it is, how it works, why you chose it.
2. Add a short real story: "I hit this problem, so I did this."
3. Claim only what you built. For anything else say: "I haven't implemented it yet, but I would approach it like this."
4. End project explanations with your improvement plan.